

Initial Project Plan / SDS

Created - April 8th, 2026

Initial deliverable last edited – April 17th, 2026

Final deliverable last edited – May 4th, 2026

Authors: Dennis Hulett, Jesse Litwin, Dacian Rapolla, Jack Sedillos

This document contains our initial and final project plan/SDS deliverable for 422's first project. It describes our plan to divide the work as well as an overview of our proposed product that we will be working on according to the guidelines given by the instructors.

Initial Roles:

- Jack: Backend
- Jesse: Backend(flask/API)
- Dennis: Database
- Dacian: Frontend(GUI)

Communicator/Coordinator: Dacian

Final contributions:

- Jack: Backend/graph generation
- Jesse: Backend(flask/API/integration of database and frontend to flask)
- Dennis: Database/Admin frontend
- Dacian: Frontend(GUI)/integrating graph generation

Project Plan

- A rationale for the build plan. Why have you broken the system into these parts, and why have you chosen these particular steps to build the system? What are your risks and your risk reduction strategies?

We organized our group based on what everyone was most comfortable/wanting to work on while still trying to maintain a workload balance – we tried to divide the work about 25% given that criteria and roughly stuck to it. We made sure to communicate frequently; we had bi-weekly meetings every week – 9am-10am on Tuesdays and Thursdays in the EMU (Erb Memorial Union) – since the project started, where we discussed our progress, what still needed to be done, and any questions or concerns regarding our own or anyone else's work. Outside of in-person meetings, we communicated primarily through a text group chat, where we

coordinated repository pushes, any extraneous meeting times, such as 3rd May and 4th May 2026, where we met in the science library around noon to discuss our documentation and final cleanup to ensure our product was ready and met the requirements by the deadline. We also sent more technical updates in a Discord server, such as the database document structure, that were too lengthy or needed specific formatting and would not be fit to send over a group-chatlog.

Our Breakdown:

- Gather all data from sources provided (MILESTONE: Completed by project plan due date, or 17th april, by all members of group)
 - the project handouts on the course website and provided csv files
- Gather needed libraries (MILESTONE: Completed by following Tuesday, or 21st April, by all members of group)
 - Matplotlib, Fask, pymongo, basic python libraries, etc.
- Store data in a mongoDB
 - Document structure to best match data provided in csv files (MILESTONE: Completed 21st April, by Dennis)
 - Parsing csv files given to the system by admin (MILESTONE: Completed 31st April, by Dennis)
- Create a general frontend (Completed 26th April, by Dacian)
 - Select a user class: admin or student. (MILESTONE: Completed 4th May, by Jesse)
 - Student: Select degree, time, then classes and instructors based on graph distributions (MILESTONE: Completed 29th April, by Dacian)
 - Admin: Update system data (MILESTONE: Completed 30th April, by Dennis)
 - graphs for grade distribution for each instructor of each course shown to aid in decision making.(matplotlib or pandas) (MILESTONE: Completed 27th April, by Jack)
- Establish an API connection with flask (MILESTONE: Completed 30th April, by Jesse and Jack)
- Develop a backend to talk to the API (MILESTONE: Completed 3rd may, by Jesse and Jack)
- Create all functions that users will need for grabbing their classes (Completed as other things were checked off, by all members of group)
- Have established an MVP at this point with minimal requirements met (MILESTONE: Completed 3rd May, by all members of group)
- Create installation package + ensure it is working (MILESTONE: Completed 4th May, by all members of group)
- Create proper documentation to match with our code and project requirements (MILESTONE: Completed 4th may, by all members of group)

Monitoring and reporting was tracked through Github, text, and Discord, as described above. We sent a text to the group each time one of us wanted to push to our shared repository to ensure there would be no merge conflicts, and had everyone confirm what was being

pushed/added/changed each time. We wrote the milestones to be completed in order so it represents a general build plan, but some things naturally happened out of order based on when we were best able to get our work done. Our initial plan was to complete the API/Flask by April 20th, the MVP by April 26th and the entire system complete by May 2nd, giving ourselves a few days of slack time to account for slippage. This ended up being very necessary, as we all got hit hard by midterms and it caused a delay in fully completing all of our systems for our product.

Initially the decisions for build order came relatively easy since we all have technical experience and understand to some degree the kinds of things we would like to have in place while designing other parts; we wanted the data to be stored in a DB and a basic frontend to be designed prior to designing a flask api connection for testing purposes, and the frontend can be built subsequently to the database because it is unrelated. Our thought was that after a flask script has been written, the python backend could be built with appropriate hooks, though we may edit/change both the backend and flask script as we go. We decided to build the installation package at the very end so that we are not rewriting anything due to future changes. Along the way we risk not including all of the correct features in our frontend, but we reduce this risk by always reflecting back onto the requirements and our design approach for fixes and optimization.

In the end we all built our systems essentially simultaneously, with the database structure being finalized first to allow backend to know how to pull and format data from the database, then frontend to allow backend to integrate, then combining the admin and student interfaces into a final product. Building everything simultaneously ended up working in our favor – it made integration easier as we were constantly communicating our updates and progress to each other as we went and allowed everyone to have the time they needed to finish their systems. Our installation package was still built last as we specified to ensure there would be no conflicts while building the system.

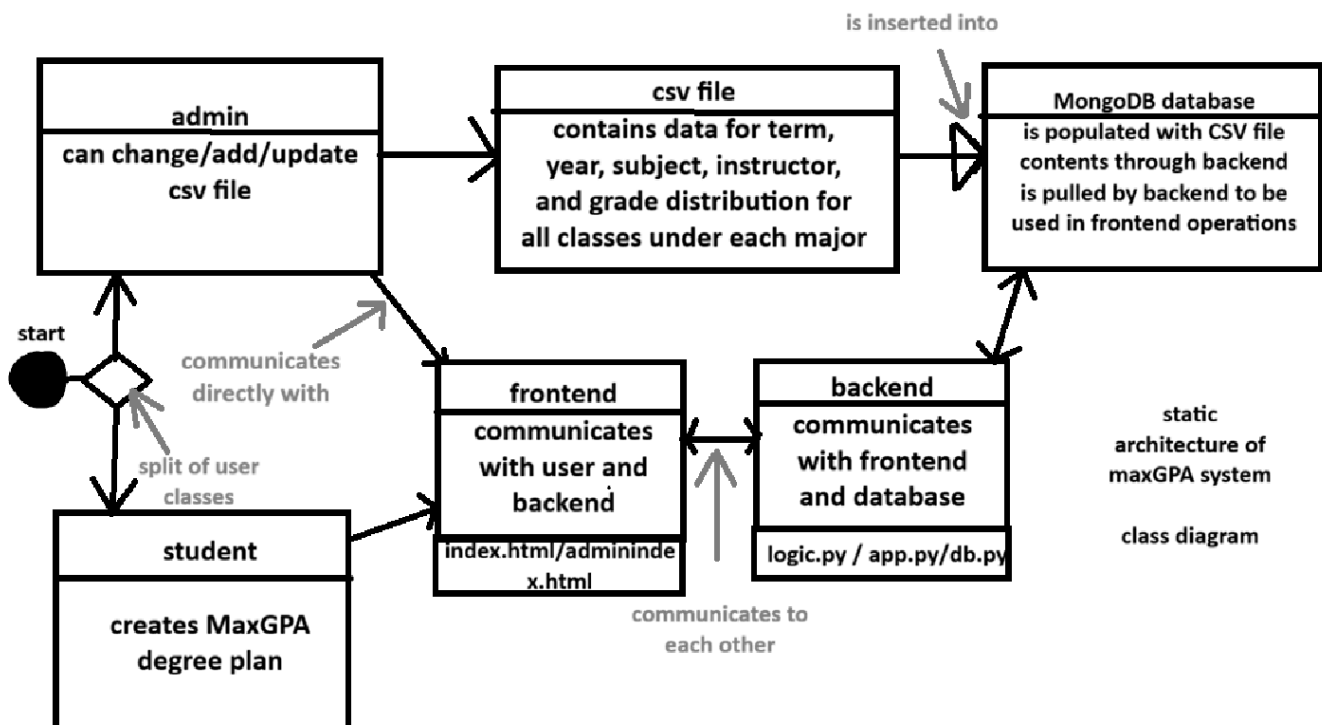
SDS

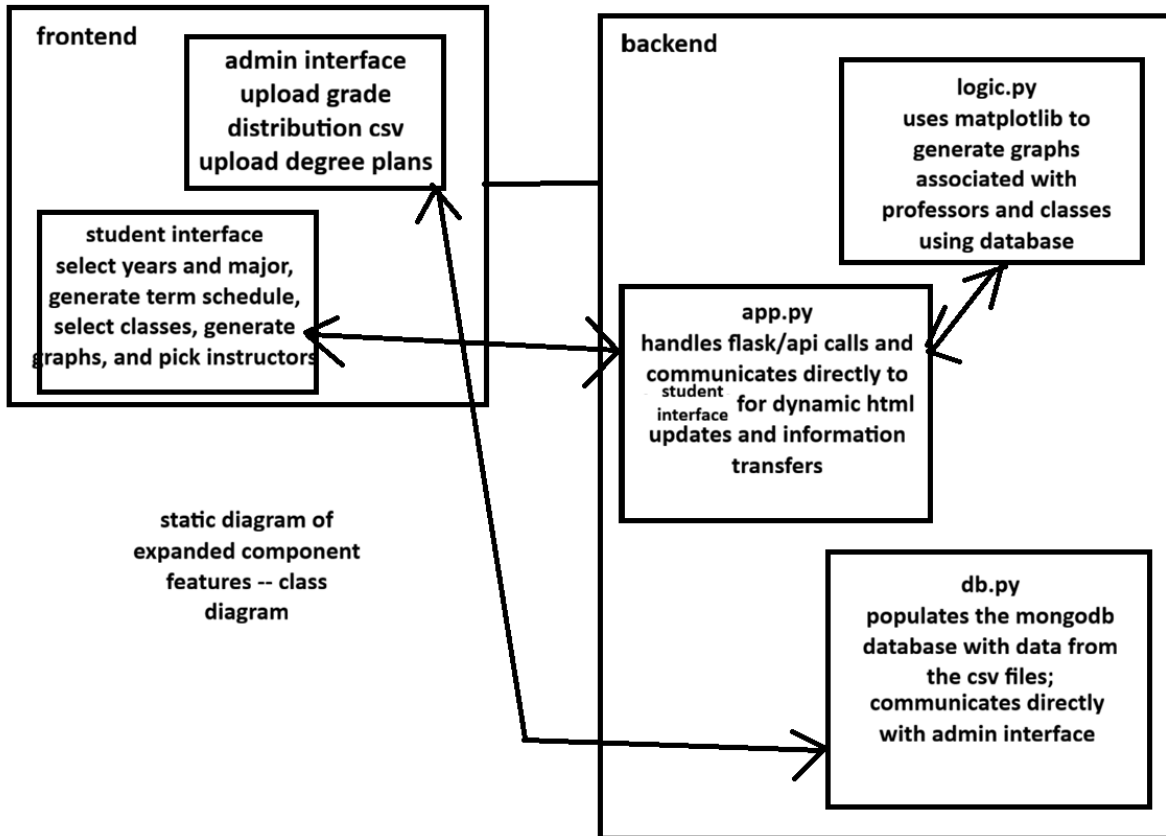
DESCRIPTION OF PRODUCT:

- MaxGPA is a program that will help students maximize their GPA.
- The user of the system has two choices – either they are an admin or a student.
 - The admin will be prompted to submit a .csv file containing the UO course and grade distributions to populate the database that will be pulled from for the student interface. The .csv uploaded should meet the standards given by typical UO grade distribution .csv files. The admin can also select a different tab to upload three major degree path .csv files, which will be provided by us in the installation folder, and are put into the database and used to organize the student interface.
 - The student user will initially be prompted with a selection of majors to choose from. Once picked, they will then choose a window of time from the available years. A blank graduation plan with rows and columns is generated with respect

to the window of time selected. The user will then be able to select from a dropdown menu at each term, various required courses that have yet to be selected for the major. Once selected, the course is omitted from the selection list to prevent duplicate courses, and added to the graduation plan. A user will be able to select 4 courses for each term. Once a course is added to the graduation plan, a side panel of graph GPA distributions for each instructor of the course appears, and the user can choose a specific instructor based on the graphs and lock it into the term's schedule.

SYSTEM OVERVIEW/SOFTWARE ARCHITECTURE:

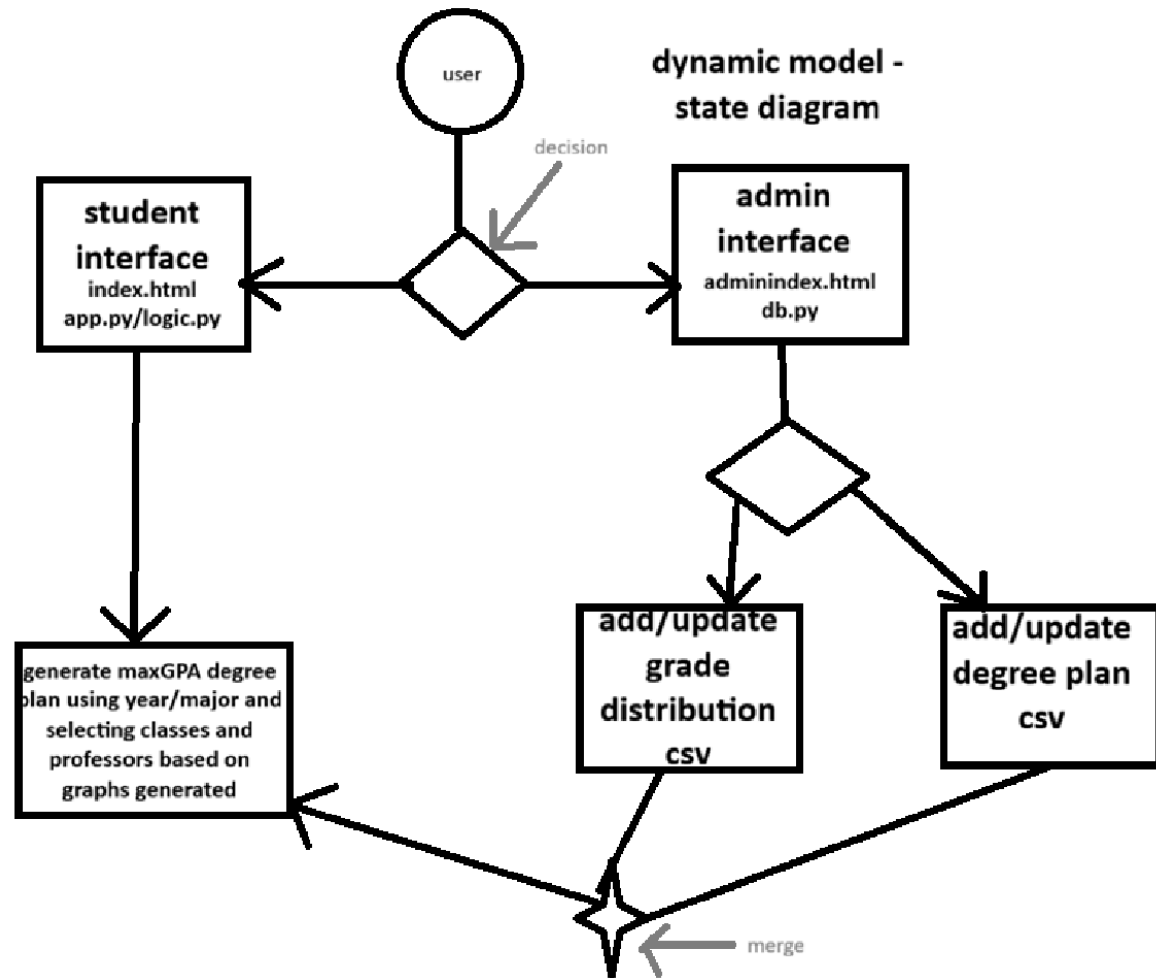




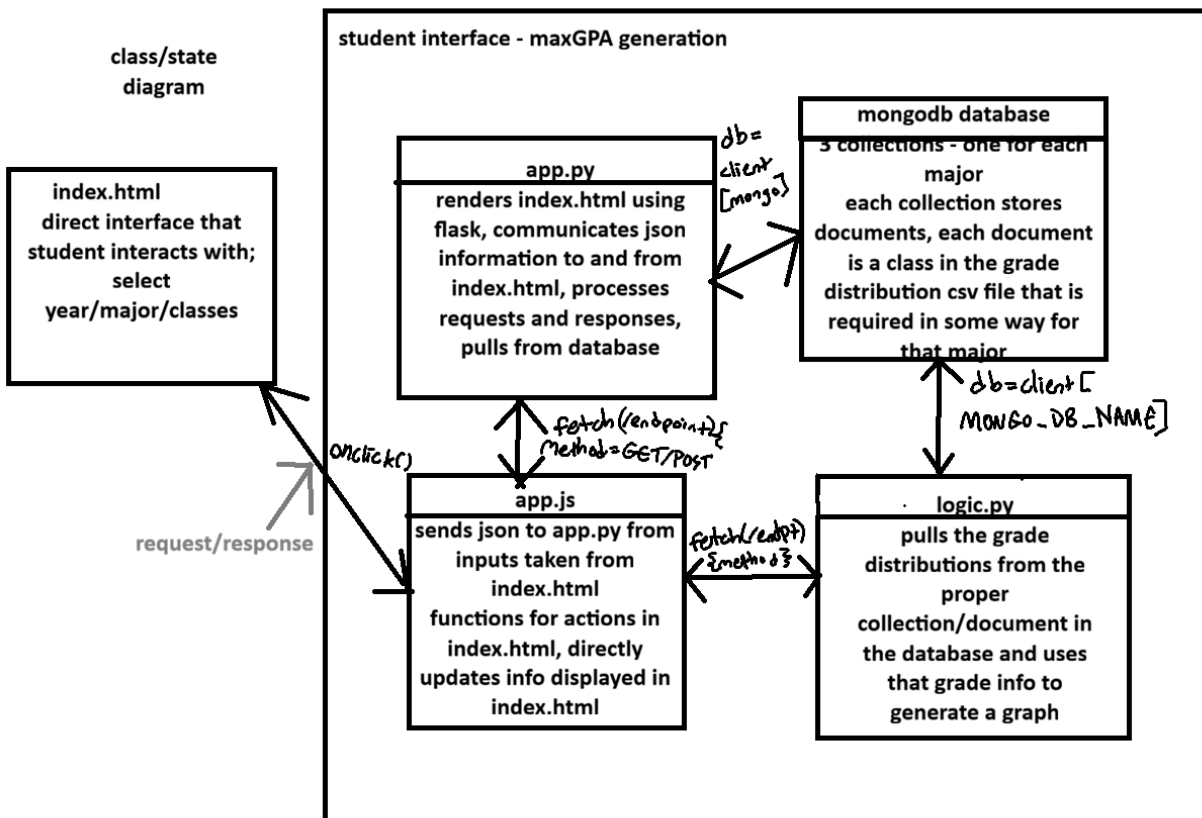
Summary: two class diagrams of the maxGPA system. From the start you are either an admin user class or a student user class. Admin has direct access to our frontend, as well as the csv file. Admin communicates directly with frontend by uploading the csv, which then gets sent to backend, which then populates the database. The database is pulled by backend and sent to frontend for our student interface, seen by the student user class, who uses it to create maxGPA degree plans.

We created a class diagram for our initial submission (that has now been refined) to get the best abstract overview of our systems/classes and how they would connect to each other. We have also created a larger diagram of just how “frontend” and “backend” communicate with each other, and more detailed descriptions of the sub-modules within those larger modules.

For a written object diagram of the database structures, refer to README_Dennis.txt, located in the Database folder in the maxGPA project folder.



Summary: a state diagram of our maxGPA system. The user chooses which interface to interact with and then interacts with that interface according to their purpose in launching the system. Once the admin is done updating/inserting the csv they can choose to interact with the user interface, thus the merge.



Summary: a class/state diagram of our student interface, with each component having a detailed description as well as a trigger between two modules that would connect them to one another.

ALTERNATIVE DESIGNS:

We considered alternate designs for the database collections – currently there are four: CS/BA/MATH collections to store all class data required for one major in one place and a fourth degreeplan collection to store degreeplan csv data, but we had a pitch to instead have each collection be a subset of data (eg. one collection for professors and grade dicts, one for classes/nums, etc.). We ended up sticking with the current system because it seemed easier to organize the data indiscriminately in one place.

Our original studentside html design was also different – originally there were going to be expandable dropdowns for every term, very similar to the paper copy of the project 1 handout given out in class; We decided it was a bit too cumbersome and took up a lot of space where our current model is just a grid you select classes into.

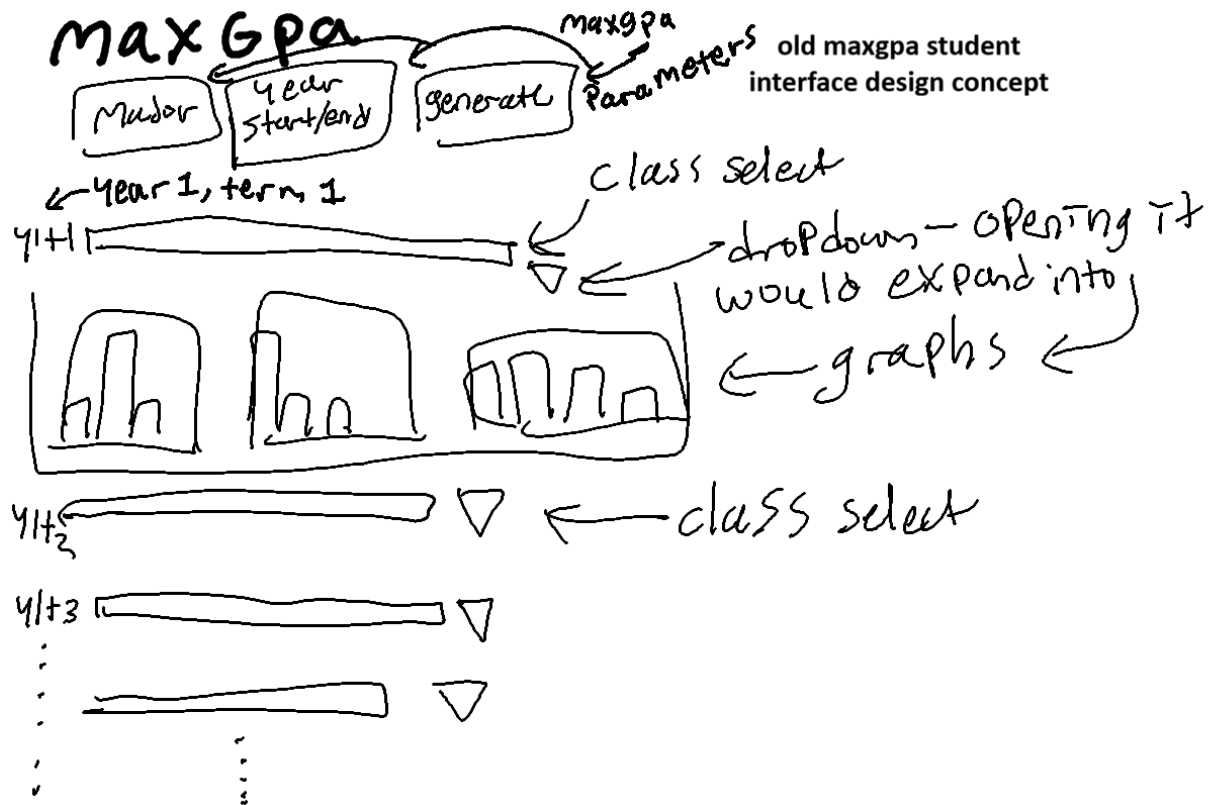


Diagram of old maxgpa student interface design

6. References

<https://classes.cs.uoregon.edu/26S/cs422/Handouts/Template-SDS.pdf>

Faulk, Stuart. (2011-2017). CIS 422 Document Template. Downloaded from <https://uocis.assembla.com/spaces/cis-f17-template/wiki> in 2018. It appears as if some of the material in this document was written by Michal Young.

IEEE Std 1016-2009. (2009). IEEE Standard for Information Technology—Systems Design—

Software Design Descriptions. <https://ieeexplore.ieee.org/document/5167255>

Parnas, D. L. (1972). On the criteria to be used in decomposing systems into modules. Commun.

ACM, 15(12), 1053-1058.

7. Acknowledgements

This template builds slightly on a similar document produced by Stuart Faulk in 2017, and heavily on the publications cited within the document, such as IEEE Std 1016-2009; and the final product builds heavily from the template provided by A. Hornof on the CS 422 class website.

All diagrams in this document were made by Dennis Hulett.